

第 11 节 实操任务 1 实现Reviewer-Agent

学习目标：review_node 能对知识条目进行四维度 LLM 评分，输出结构化 JSON
目标文件：workflows/nodes.py (review_node 正式版)

背景

V3 的核心是“反馈驱动的质量闭环”——Reviewer 评估，Reviser 修改，循环到通过。
本实操升级 Reviewer 节点的实现（拓扑仍是 10-2 的 5 节点版）。

核心原则：只评估不修改（Evaluate, don't modify）。

步骤 1：用 AI 编程工具生成 review_node 正式版

以下代码可以用 OpenCode、Claude Code、Cursor、Trae 或通义灵码等任意 AI 编程工具生成。

提示词：

```
1 请帮我编写 workflows/reviewer.py 中的 review_node 函数：
2
3 需求：
4 1. Reviewer 审核的对象是 state["analyses"]（不是 articles, articles 在 organ
5 2. 5 维度评分，每维 1-10 分，权重如下：
6     - summary_quality（摘要质量）：25%
7     - technical_depth（技术深度）：25%
8     - relevance（相关性）：20%
9     - originality（原创性）：15%
10    - formatting（格式规范）：15%
11 3. 用代码重算加权总分（不要信任模型算术）
12 4. 加权总分 >= 7.0 为通过
13 5. 只审核前 5 条 analyses（控 token 消耗）
14 6. temperature=0.1（评分一致性）
15 7. LLM 调用失败时自动通过（不阻塞流程）
16 8. 返回 {review_passed, review_feedback, iteration, cost_tracker}
17
18 依赖：
19 - chat_json(prompt, system=..., temperature=...) 返回 (parsed_json, usage
20 - accumulate_usage(tracker, usage)
21 - KBState 的 plan, analyses, iteration, cost_tracker 字段
```

生成的代码（参考实现）：

```

1 # 权重字典：写在代码里，方便不改 prompt 调整权重
2 REVIEWER_WEIGHTS = {
3     "summary_quality": 0.25,
4     "technical_depth": 0.25,
5     "relevance": 0.20,
6     "originality": 0.15,
7     "formatting": 0.15,
8 }
9 REVIEWER_PASS_THRESHOLD = 7.0
10
11
12 def review_node(state: KBState) -> dict:
13     """Reviewer 节点：对 analyses 进行 5 维度质量审核
14
15     核心原则：只评估不修改（Evaluate, don't modify）。
16     Reviewer 看到的是 Analyzer 输出的 analyses，不做任何改动，只给分 + 反馈。
17     """
18     analyses = state.get("analyses", [])
19     iteration = state.get("iteration", 0)
20     tracker = state.get("cost_tracker", {})
21
22     if not analyses:
23         return {
24             "review_passed": True,
25             "review_feedback": "没有条目需要审核",
26             "iteration": iteration + 1,
27         }
28
29     # 只审核前 5 条，控制 token 消耗
30     sample = analyses[:5]
31
32     prompt = f"""你是知识库质量审核员。请审核以下分析结果：
33
34 {json.dumps(sample, ensure_ascii=False, indent=2)}
35
36 请按以下维度评分（每项 1-10 分）：
37 1. summary_quality - 摘要质量
38 2. technical_depth - 技术深度
39 3. relevance - 相关性
40 4. originality - 原创性
41 5. formatting - 格式规范
42
43 请用 JSON 格式回复：
44 {{
45     "scores": {{
46         "summary_quality": 8,
47         "technical_depth": 6,
48         "relevance": 9,

```

```

49         "originality": 5,
50         "formatting": 8
51     }},
52     "feedback": "具体的改进建议（指出弱项）",
53     "weak_dimensions": ["technical_depth", "originality"]
54 }
55
56 当前是第 {iteration + 1} 次审核。"""
57
58     try:
59         result, usage = chat_json(
60             prompt,
61             system="你是严格但公正的知识库质量审核员。给出具体、可操作的反馈。",
62             temperature=0.1, # 低温度保证评分一致性
63         )
64         tracker = accumulate_usage(tracker, usage)
65
66         # 【关键设计】用代码重算加权总分，不信任模型算术
67         scores = result.get("scores", {})
68         weighted_total = sum(
69             scores.get(dim, 0) * w for dim, w in REVIEWER_WEIGHTS.items()
70         )
71         weighted_total = round(weighted_total, 2)
72         passed = weighted_total >= REVIEWER_PASS_THRESHOLD
73
74         feedback = result.get("feedback", "")
75         weak_dims = result.get("weak_dimensions", [])
76         if weak_dims:
77             feedback = f"[弱项: {' '.join(weak_dims)}] {feedback}"
78
79         print(
80             f"[Reviewer] 加权总分: {weighted_total}/10, "
81             f"通过: {passed} (第 {iteration + 1} 次审核)"
82         )
83
84     except Exception as e:
85         passed = True
86         feedback = f"审核 LLM 调用失败: {e}, 自动通过"
87         print(f"[Reviewer] 审核失败, 自动通过: {e}")
88
89     return {
90         "review_passed": passed,
91         "review_feedback": feedback,
92         "iteration": iteration + 1,
93         "cost_tracker": tracker,
94     }
95
96 ---

```

步骤 2：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

```
请解释 review_node 的设计决策：
1. 为什么 Reviewer 审 analyses 而不是 articles？
2. 为什么权重字典 REVIEWER_WEIGHTS 写在代码里而不是 prompt 里？
3. 为什么用代码重算加权总分，而不是让 LLM 自己算？
4. 为什么 temperature=0.1？
5. 为什么 LLM 调用失败时自动通过而不是报错？
```

关键设计解读：

设计点	为什么这样做
只审核前 5 条	控制 token 消耗，避免长上下文降低审核质量
失败自动通过	审核是锦上添花，不能因为审核模块故障阻塞整个流水线
强制通过在 review 内	should_continue 只做简单路由判断，业务逻辑集中在节点内
temperature=0.2	审核需要高度一致性，多次审核同一内容应得到相近分数

步骤 3：确保 graph.py 使用 review_node

workflows/graph.py 应该在图里注册 review_node（v3 教师版已经配好）：

```
1 from workflows.reviewer import review_node
2
3 graph.add_node("review", review_node)
4 graph.add_edge("analyze", "review") # 分析完后进入审核
5
6 ---
```

步骤 4：运行验证

```
1 cd ~/ai-knowledge-base/v3-multi-agent
2 python3 -m workflows.graph
```

当前拓扑仍是 10-2 的 5 节点直觉版（review_node 内部从测试版换回 LLM 真版）：

```
1 collect → analyze → organize → review →[cond]→ save / organize 自循环
```

期望输出 A，首次审核通过（理想路径）：

```
1 [Collector] 采集到 10 条原始数据
2 [Analyzer] 完成 10 条分析
3 [Organizer] 整理出 10 条知识条目（迭代 0）
4 [Reviewer] 加权总分：7.2/10，通过：True（第 1 次审核）
5 [Save] 已写入 10 篇到磁盘
```

期望输出 B，首次未过 · organize 自循环修正（10-2 拓扑核心机制）：

```
1 [Collector] 采集到 10 条原始数据
2 [Analyzer] 完成 10 条分析
3 [Organizer] 整理出 10 条知识条目 (迭代 0)
4 [Reviewer] 加权总分: 6.3/10, 通过: False (第 1 次审核)
5 [Organizer] 整理出 8 条知识条目 (迭代 1)          ← review_feedback 触发 organ
6 [Reviewer] 加权总分: 7.5/10, 通过: True (第 2 次审核)
7 [Save] 已写入 8 篇到磁盘
```

⚠ 不会看到的输出 (这些是后续节才会出现的) :

```
[Planner] ... · 11-3 加 planner 节点后才有
[Reviser] ... · 11-2 加 reviser 节点后才有
[HumanFlag] ... · 11-2 加 human_flag 节点后才有
如果你看到了这些 · 说明你的 graph.py 已经是最终版 (7 节点) · 不是本节阶段
(5 节点) 的代码。
```

验证清单:

检查项	期望	实际
审核输出包含 overall_score	是	
审核输出包含 review_feedback	是	
review_passed 是 bool	是	
iteration 正确递增 (review_node 内部 +1)	是	
首次未过会触发 organize 自循环	是	
max iteration (>=2 / >=3) 触发强制通过	是	

步骤 5: 提交到 Git

```
1 git add workflows/reviewer.py
2 git commit -m "feat: implement review_node with 5-dim weighted scoring"
3
4 ---
```

完成! Reviewer 现在能对 analyses 做 5 维度加权评分, 给出结构化反馈。下节实操 2 实现 Reviser, 两者组成完整的审核修正循环。